



Xi'an Jiaotong-Liverpool University

西交利物浦大學

**INT305 Machine Learning**

**Lecture 3**

**Linear Classifiers, Logistic Regression, Multiclass Classification**

**Sichen Liu**

**Department Intelligence Science**

**[Sichen.Liu@xjtlu.edu.cn](mailto:Sichen.Liu@xjtlu.edu.cn)**

## Overview

---

- **Classification**: predicting a discrete-valued target 离散的, 代表类别是固定的, 有限的选项
  - **Binary classification**: predicting a binary-valued target 二选一  
二分类
  - **Multiclass classification**: predicting a discrete( $>2$ )-valued target 多选一  
多元分类
- Examples of binary classification
  - Predict whether a patient has a disease, given the presence or absence of various symptoms
  - Classify e-mails as spam or non-spam
  - Predict whether a financial transaction is fraudulent

# Overview

---

## Binary linear classification

- **Classification:** given a  $D$ -dimensional input  $\mathbf{x} \in \mathbb{R}^D$  predict a discrete-valued target
- **Binary:** predict a binary target  $t \in \{0,1\}$ 
  - Training examples with  $t = 1$  are called **positive examples**, and training examples with  $t = 0$  are called **negative examples**. Sorry.
  - $t \in \{0,1\}$  or  $t \in \{-1, +1\}$  is for computational convenience.
- **Linear:** model prediction  $y$  is a linear function of  $\mathbf{x}$ , followed by a threshold  $r$ :

$$\star \left\{ \begin{array}{l} z = \mathbf{w}^\top \mathbf{x} + b \\ y = \begin{cases} 1 & \text{if } z \geq r \\ 0 & \text{if } z < r \end{cases} \end{array} \right.$$

# Simplifications

## Eliminating the threshold

- We can assume without loss of generality (WLOG) that the threshold  $r = 0$ :

$$\mathbf{w}^T \mathbf{x} + b \geq r \iff \mathbf{w}^T \mathbf{x} + \underbrace{b - r}_{\triangleq w_0} \geq 0$$

可以将阈值  $r$  和 偏置项 (bias)  $b$  融合在一起, 从而让阈值固定为 0

## Eliminating the bias

不用单独考虑  $r$ .  $(b-r) = b'$

- Add a dummy feature  $x_0$  which always takes the value 1. the weight  $w_0 = b$  is equivalent to a bias (same as linear regression)

为了消除偏置项  $b$ , 要融入到  $w^T x$  中  
对  $w^T x$  进行扩维.

## Simplified model

- Receive input  $\mathbf{x} \in \mathbb{R}^{D+1}$  with  $x_0 = 1$ :

新增一个特征  $x_0$ , 让其永远为 1.  
新增一个权重向量  $w_0$ , 让其永远为  $b'$

这样, 原来的公式  $(w_1 x_1 + \dots + w_D x_D)$

$w^T x + b \geq 0$  简化为了一个  $D+1$  维的

公式

$$z = \mathbf{w}^T \mathbf{x}$$

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

$$\Rightarrow (w_0 x_0 + w_1 x_1 + \dots + w_D x_D)$$

## Examples

---

- Let's consider some simple examples to examine the properties of our model
- Let's focus on minimizing the training set error, and forget about whether our model will generalize to a test set. 先关注训练集, 暂时忽略测试集

## Examples

---

**NOT** 异或, 不同为1, 相同为0

| $x_0$ | $x_1$ | $t$ |
|-------|-------|-----|
| 1     | 0     | 1   |
| 1     | 1     | 0   |

- Suppose this is our training set, with the dummy feature  $x_0$  included.
- Which conditions on  $w_0, w_1$  guarantee perfect classification?
  - When  $x_1 = 0$ , need:  $z = w_0x_0 + w_1x_1 \geq 0 \iff w_0 \geq 0$
  - When  $x_1 = 1$ , need:  $z = w_0x_0 + w_1x_1 < 0 \iff w_0 + w_1 < 0$
- Example solution:  $w_0 = 1, w_1 = -2$
- Is this the only solution? 但是有无穷解

## Examples

---

AND

| $x_0$ | $x_1$ | $x_2$ | $t$ |
|-------|-------|-------|-----|
| 1     | 0     | 0     | 0   |
| 1     | 0     | 1     | 0   |
| 1     | 1     | 0     | 0   |
| 1     | 1     | 1     | 1   |

$$z = w_0x_0 + w_1x_1 + w_2x_2$$

$$\text{Need: } w_0 < 0$$

$$\text{Need: } w_0 + w_2 < 0$$

$$\text{Need: } w_0 + w_1 < 0$$

$$\text{Need: } w_0 + w_1 + w_2 \geq 0$$

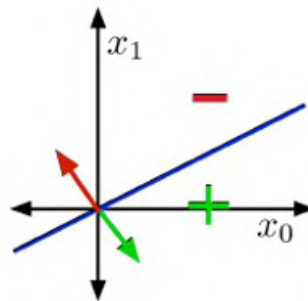
Example solution:  $w_0 = -1.5, w_1 = 1, w_2 = 1$

## The Geometric Picture

点是数据  $x$ , 权重  $w$  定义了一条线 (决策边界)

Input Space, or Data Space for NOT example

输入空间



| $x_0$ | $x_1$ | $t$ |
|-------|-------|-----|
| 1     | 0     | 1   |
| 1     | 1     | 0   |

- Training examples are points
- Weights (hypotheses)  $w$  can be represented by **half-spaces**

$$H_+ = \{x: w^T x \geq 0\}, H_- = \{x: w^T x < 0\}$$

➤ The boundaries of these half-spaces pass through the origin (why?)

因为 origin is (0,0). 也就是  $(x_0, x_1) = (0, 0)$   
 $w_0 \cdot 0 + w_1 \cdot 0 = 0$

- The boundary is the **decision boundary**:  $\{x: w^T x = 0\}$

➤ In 2-D, it's a line, but in high dimensions it is a hyperplane

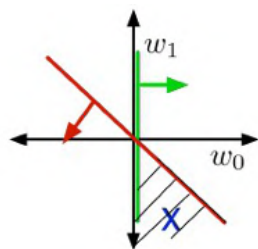
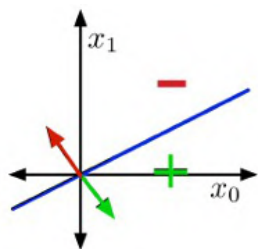
- If the training examples can be perfectly separated by a linear decision rule, we say **data is linearly separable**.

## The Geometric Picture

Weight Space 权重空间

点是权重  $w$  (每一点都代表一种可能的决策方案)

数据  $x$  会定义一条线, 对权重  $w$  的取值范围给出限制



$$w_0 \geq 0$$

只需要确保  $w_0$  为正

$$w_0 + w_1 < 0$$

需要确保  $w_0 + w_1$  为负, 两者都从负域取值

- Weights (hypotheses)  $w$  are points
- Each training example  $x$  specifies a half-space  $w$  must lie in to be correctly classified:

$$w^T x \geq 0 \text{ if } t = 1.$$

针对  $x = [1, 0]$ , 正确答案  $t = 0$ ,  $w^T x < 0$

- For NOT example:

$$\triangleright x_0 = 1, x_1 = 0, t = 1 \implies (w_0, w_1) \in \{w: w_0 \geq 0\}$$

$w_0 + w_1 < 0$ ; 对应红线  $w_0 + w_1 = 0$  作为边界, 所有可行的  $w$  点都必须在这

$$\triangleright x_0 = 1, x_1 = 1, t = 0 \implies (w_0, w_1) \in \{w: w_0 + w_1 < 0\}$$

条线左下方,  $x_1, x_2$  只能选负值.

- The region satisfying all the constraints is the **feasible region**; if this region is nonempty, the problem is **feasible**, otw it is **infeasible**.

针对  $x = [1, 0]$ , 正确答案  $t = 1$ ,  $w^T x$  必须  $\geq 0$ . 也就是  $w_0 \geq 0$ ; 这表明对于任何  $w = [w_0, w_1]$  其中第一个参数  $w_0$  必须大于等于 0. 在权重空间中, 这就画出了一条竖线  $w_0 = 0$  作为边界, 并规定所有可行的  $w$  点都在绿线右边

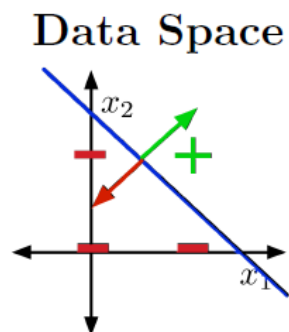
## The Geometric Picture

---

- The **AND** example requires three dimensions, including the dummy one.
- To visualize data space and weight space for a 3-D example, we can look at a 2-D slice.
- The visualizations are similar.
  - Feasible set will always have a corner at the origin.

# The Geometric Picture

Visualizations of the **AND** example



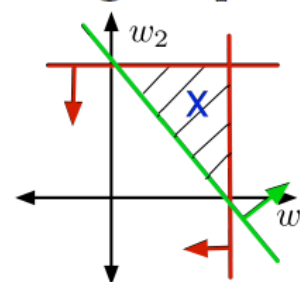
在  $x_0=1$  的平面上

- Slice for  $x_0 = 1$  and
- Example sol:  $w_0 = -1.5, w_1 = 1, w_2 = 1$
- Decision boundary:

$$w_0x_0 + w_1x_1 + w_2x_2 = 0$$

$$\implies -1.5 + x_1 + x_2 = 0$$

Weight Space



$w_0 = -1.5$  的平面上

- Slice for  $w_0 = -1.5$  for the constraints

$$w_0 < 0$$

红线 {

$$w_0 + w_2 < 0 \quad -1.5 + w_2 < 0 \implies w_2 < 1.5$$

$$w_0 + w_1 < 0 \quad -1.5 + w_1 < 0 \implies w_1 < 1.5$$

绿线 •

$$w_0 + w_1 + w_2 \geq 0 \quad -1.5 + w_1 + w_2 \geq 0$$

$$\implies w_1 + w_2 \geq 1.5$$

## Summary | Binary Linear Classifiers

---

- **Summary:** targets  $t \in \{0,1\}$ , inputs  $\mathbf{x} \in \mathbb{R}^{D+1}$  with  $x_0 = 1$ , and model is defined by weights  $\mathbf{w}$  and

$$z = \mathbf{w}^\top \mathbf{x}$$

$$y = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

感知机学习

线性规划

- How can we find good values for  $\mathbf{w}$ ?
- If training set is linearly separable, we could solve for  $\mathbf{w}$  using linear programming
  - We could also apply an iterative procedure known as the *perceptron algorithm* (but this is primarily of historical interest).
- If it's not linearly separable, the problem is harder
  - Data is almost never linearly separable in real life.

## Towards Logistic Regression

---

Towards Logistic Regression 逻辑回归

## Loss Functions 损失函数

当不能完美线性分化时, 要找一个尽可能好的  $w$ . 损失函数的作用就是衡量  $w$  好与不好的标准

- Instead: define loss function then try to minimize the resulting cost function

➤ Recall : cost is loss averaged (or summed) over the training set

- Seemingly obvious loss function: 0-1 loss

如果模型预测  $y$  和真实标签  $t$  相等 ( $y=t$ )

预测正确, 损失是 0

如果模型预测  $y$  和真实标签  $t$  不相等 ( $y \neq t$ ), 预测失败, 损失为 1.

$$\mathcal{L}_{0-1}(y, t) = \begin{cases} 0 & \text{if } y = t \\ 1 & \text{if } y \neq t \end{cases} \\ = \mathbb{I}[y \neq t]$$

这样的话, 虽然很容易根据 0-1 loss 之后的损失总和直接得到错误样本的数量。但是因为其不可导, 难以用高效的优化算法来找到最小的  $w$ .

loss function 损失函数: 针对单个样本的, 衡量我们的模型在这个数据上预测的有多差

cost function 代价函数: 针对整个训练集的, 通常是单个样本损失的总和或平均值.

loss 相当于试卷中的某一道题是否回答正确

cost 相当于整个试卷的总扣分

## Attempt 1: 0-1 loss

---

Cost, 对所有样本的损失总和求平均值.

- Usually, the cost  $\mathcal{J}$  is the averaged loss over training examples; for 0-1 loss, this is the **misclassification rate**:

$$\mathcal{J} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y^{(i)} \neq t^{(i)}] = \frac{\text{错误分类的样本数量}}{\text{总样本数量}}$$

## Attempt 1: 0-1 loss

---

但是 0-1 loss 难以优化.

- Problem: how to optimize? In general, a hard problem (can be NP-hard)
- This is due to the step function (0-1 loss) not being nice (continuous/smooth/convex etc)

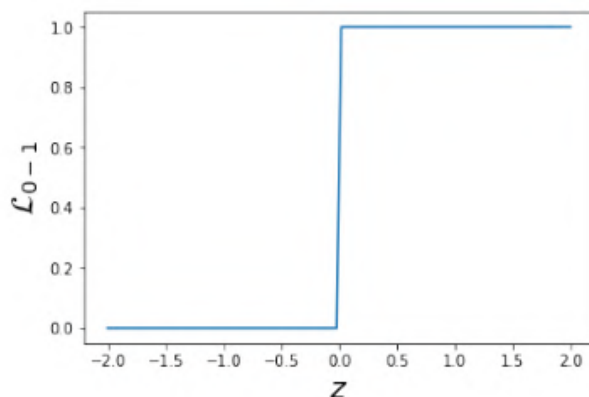
## Attempt 1: 0-1 loss

- Minimum of a function will be at its critical points.
- Let's try to find the critical point of 0-1 loss.
- Chain rule:

$$\frac{\partial \mathcal{L}_{0-1}}{\partial w_j} = \frac{\partial \mathcal{L}_{0-1}}{\partial z} \frac{\partial z}{\partial w_j}$$

- But  $\partial \mathcal{L}_{0-1} / \partial z$  is zero everywhere it's defined!

因为 0-1 loss  
非黑即白, 只有  
=0 和 ≠0 两种  
情况



- $\partial \mathcal{L}_{0-1} / \partial w_j = 0$  means that changing the weights by a very small amount probably has no effect on the loss.
- Almost any point has 0 gradient! 斜率

导数为0的点

通常方程的最小值都在其 critical point 上

最小化一个代价函数  $J(w)$ , 标准做法就是去计算关于权重  $w$  的导数  $\frac{\partial J}{\partial w}$ , 并等于 0.

权重  $w \rightarrow$  分数  $z \rightarrow$  损失  $L$

先看  $w_j$  的变化如何影响分数  $z$  ( $\frac{\partial z}{\partial w_j}$ )  
再看分数  $z$  的变化如何影响损失  $L$  ( $\frac{\partial L}{\partial z}$ )  
但  $\frac{\partial L}{\partial z} = 0$ , 所以  $\frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w_j} = 0$ .

## Attempt 2: Linear Regression 线性回归

---

- Sometimes we can replace the loss function we care about with one which is easier to optimize. This is known as **relaxation** with a smooth **surrogate loss function**. 代替损失函数
- One problem with  $\mathcal{L}_{0-1}$ : defined in terms of final prediction, which inherently involves a discontinuity. 直接用连续分数  $z = \mathbf{w}^T \mathbf{x}$  来计算损失
- Instead, define loss in terms of  $\mathbf{w}^T \mathbf{x}$  directly.
  - Redo notation for convenience:  $z = \mathbf{w}^T \mathbf{x}$

## Attempt 2: Linear Regression

---

- We already know how to fit a linear regression model. Can we use this instead?

$$z = \mathbf{w}^T \mathbf{x} \quad z \text{ 是一个连续的值}$$

$$\mathcal{L}_{SE}(z, t) = \frac{1}{2} (z - t)^2 \quad \text{平方误差损失}$$

- Doesn't matter that the targets are actually binary. Treat them as continuous values.
- For this loss function, it makes sense to make final predictions by thresholding  $z$  at  $\frac{1}{2}$

(why?)

损失函数换成  $\mathcal{L}_{SE}(z, t) = \frac{1}{2}(z - t)^2$ . 把  $(z - t)$  这个差距平方. 这样既能保证损失值为正, 又能保证放大差距大的错误.

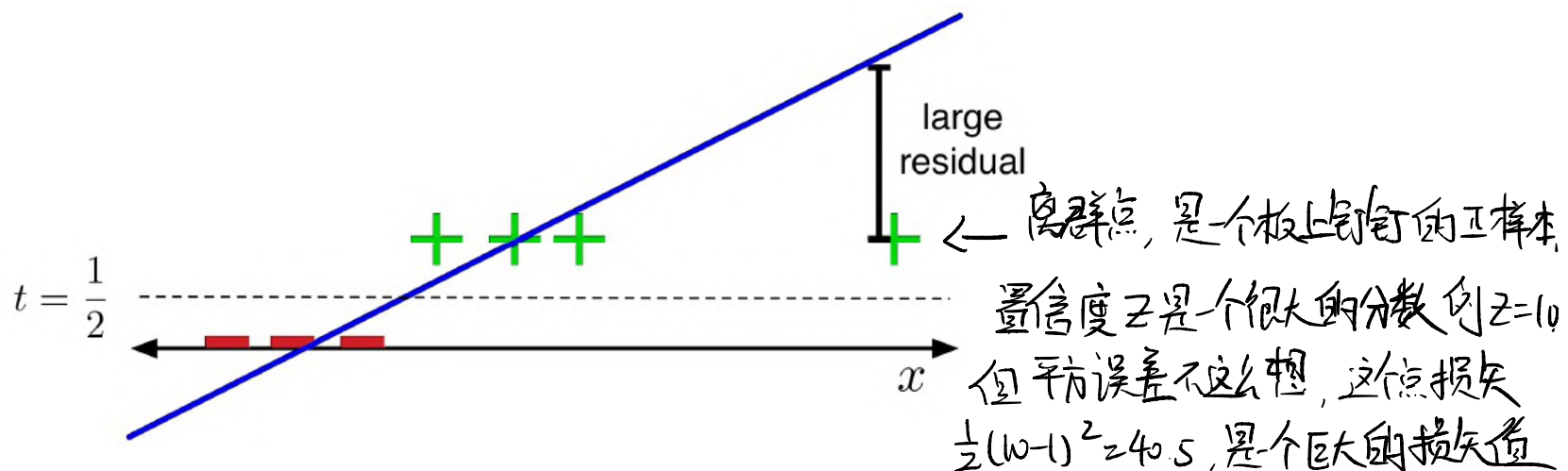
$\frac{1}{2}$  的原因是 代表正与负都是五五开.

$z$ : 实际输入与权重计算的结果

$t$ : 样本对应的标签值

## Attempt 2: Linear Regression

The problem:



- The loss function hates when you make correct predictions with high confidence!

- It  $t = 1$ , it's more unhappy about  $z = 10$  than  $z = 0$ .

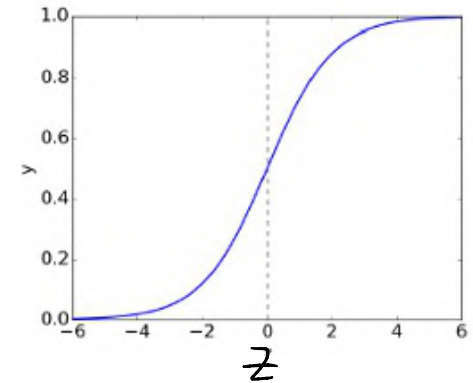
为了减小这个损失 分割线会被向  $x$  轴靠近, 原本一些靠近中间点, 可能因为斜率而被错误划分。

对于采用 linear regression 中计算  $z$  的方法, 因为  $z$  的输出范围可以是  $(-\infty, +\infty)$  区间内任意取值. 这对于只需要回答 0 或 1 的分类问题来说, 没有必要.

## Attempt 3: Logistic Activation Function

- There's obviously no reason to predict values outside  $[0, 1]$ . Let's squash  $y$  into this interval. 尝试把模型的分值限制在  $[0, 1]$  之间.
- The **logistic function** is a kind of **sigmoid**, or S-shaped function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



- $\sigma^{-1}(y) = \log(y/(1 - y))$  is called the **logit**.
- A linear model with a logistic nonlinearity is known as **log-linear**:

$$z = \mathbf{w}^T \mathbf{x}$$

$$y = \sigma(z)$$

$$\mathcal{L}_{SE}(y, t) = \frac{1}{2} (y - t)^2$$

当  $z$  趋向正无穷, 输出  $G(x)$  趋向 1  
当  $z$  趋向负无穷, 输出  $G(x)$  趋向 0.

- Used in this way,  $\sigma$  is called an **activation function**. 激活函数

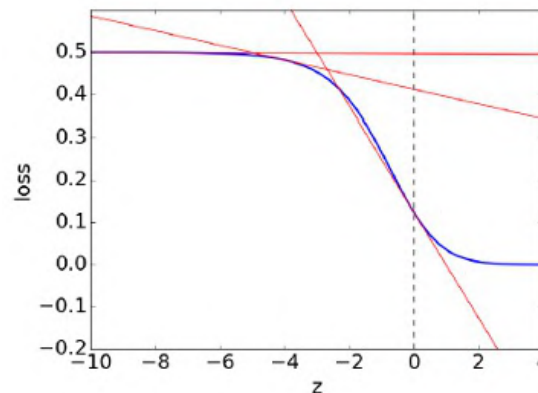
$w_{new} = w_{old} - \alpha \cdot \text{gradient}$   
如果  $z$  极大或极小会  
导致 gradient 变得无限  
接近 0, 那么  $w_{new}$  会  
无限近似于  $w_{old}$ , 那么  
就不会发生改变.

根据上图可以观察到, 当  $z$  很大时,  $y = \sigma(z)$  会无限接近于 1, 所以损失函数  $\frac{1}{2}(y - t)^2$  会无限接近于 0, 但是当  $z$  非常小的时候,  $y = \sigma(z)$  会无限接近于 0, 损失是  $\frac{1}{2}(0 - 1)^2 = 0.5$

## Attempt 3: Logistic Activation Function

### The problem:

(plot of  $\mathcal{L}_{SE}$  as a function of  $z$ , assuming  $t = 1$ )



$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial w_j}$$

当  $z$  变得越来越大, 损失曲线越来越平坦, 这意味着斜率接近 0. 如果  $\frac{\partial \mathcal{L}}{\partial z}$  趋近于 0, 那么  $\frac{\partial \mathcal{L}}{\partial z} \frac{\partial z}{\partial w_j}$  也会趋近于 0.

- For  $z \ll 0$ , we have  $\sigma(z) \approx 0$ .
- $\frac{\partial \mathcal{L}}{\partial z} \approx 0$  (check)  $\rightarrow \frac{\partial \mathcal{L}}{\partial w_j} \approx 0 \rightarrow$  derivative w.r.t.  $w_j$  is small  $\rightarrow w_j$  is like a critical point.
- If the prediction is really wrong, you should be far from a critical point (which is your candidate solution).

sigmoid 输出  $y$  在  $[0, 1]$  之间, 我们可以不把它看作一个普通分数, 而是一个**概率**  
 $y$  是模型预测真实标签  $t=1$  的概率,  $y = P(t=1|x)$ ,  $1-y$  就是预测  $t=0$  的概率.

## Logistic Regression $y = \sigma(z) = \frac{1}{1+e^{-z}}$

- Because  $y \in [0, 1]$ , we can interpret it as the estimated probability that  $t = 1$ . If  $t = 0$ , then we want to heavily penalize  $y \approx 1$ .
- The pundits who were 99% confident Clinton would win were much more wrong than the ones who were only 90% confident.
- **Cross-entropy loss** (aka log loss) captures this intuition:

交叉熵

损失

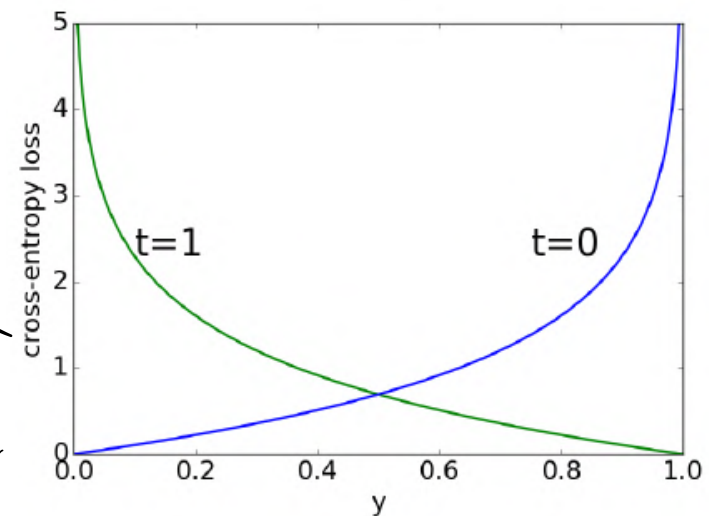
$$\mathcal{L}_{\text{CE}}(y, t) = \begin{cases} -\log y & \text{if } t = 1 \\ -\log(1 - y) & \text{if } t = 0 \end{cases}$$

$$= -t \log y - (1 - t) \log(1 - y)$$

当真实标签  $t=1$  时, 损失是  $-\log(y)$ 。预测的概率  $y$  越接近 1, 损失越小; 越接近 0, 损失就趋向于无穷大

当真实标签  $t=0$  时, 损失是  $-\log(1-y)$ 。预测的概率  $y$  越接近 0, 损失越小; 越接近 1, 损失也趋向于无穷大。

这样会惩罚那些高置信度的预测。



# Logistic Regression

Logistic regression:

$$z = \mathbf{w}^T \mathbf{x}$$

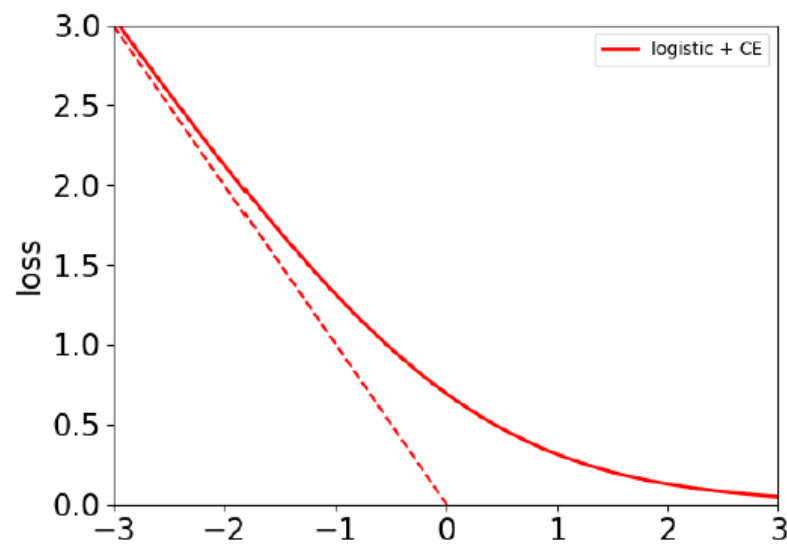
$$y = \sigma(z) \leftarrow \text{回答正确的概率}$$
$$= \frac{1}{1 + e^{-z}}$$

$$\mathcal{L}_{\text{CE}}(y, t) = -t \log y - (1 - t) \log(1 - y)$$

---

Plot is for target  $t = 1$ .

最终的损失(交叉熵)随分数 $z$ 变化的曲线.



当 $z$ 很大(正确预测)时, 损失趋近于0.

当 $z$ 很小(错误预测)时, 损失稳定增大, 曲线永远不变平.

## Gradient Descent for Logistic Regression

---

- How do we minimize the cost  $J$  for logistic regression? No direct solution.
  - Taking derivatives of  $J$  w.r.t.  $\mathbf{w}$  and setting them to 0 doesn't have an explicit solution. 逻辑回归的损失函数是一个凸函数 convex function. 只有一个最低点.
- However, the logistic loss is a **convex function** in  $\mathbf{w}$ , so let's consider the **gradient descent** method from the last lecture.
  - Recall: we **initialize** the weights to something reasonable and repeatedly adjust them in the **direction of steepest descent**.
  - A standard initialization is  $\mathbf{w} = 0$ . (why?)

通常将  $\mathbf{w}$  初始化为 0

计算  $z$ : 对于任何输入  $x$ ,  $z = \mathbf{w}^T x = 0^T x = 0$ .

计算概率  $y$ :  $y = G(z) = G(0) = \frac{1}{1+e^0} = \frac{1}{1+1} = 0.5$

这意味着在训练开始之前, 我们的模型对于任何输入都给出一个 50% 概率. 公平, 无偏见.

# Gradient of Logistic Loss 计算交叉熵损失函数关于权重 $w_j$ 的梯度 (导数), 找到下山方向

Back to logistic regression:

$$\mathcal{L}_{\text{CE}}(y, t) = -t \log y - (1 - t) \log(1 - y)$$

$$y = 1/(1 + e^{-z}) \text{ and } z = \mathbf{w}^T \mathbf{x}$$

$w_j$  如何影响分数  $z$

$z$  如何影响概率  $y$

$y$  如何影响损失  $\mathcal{L}$



Therefore

链式法则  $\frac{\partial \mathcal{L}_{\text{CE}}}{\partial w_j} = \frac{\partial \mathcal{L}_{\text{CE}}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_j} = \left( -\frac{t}{y} + \frac{1-t}{1-y} \right) \cdot y(1-y) \cdot x_j$   

$$= (y - t)x_j$$

(verify this)

Gradient descent (coordinatewise) update to find the weights of logistic regression:

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial y} = \frac{\partial (-t \log y)}{\partial y} - \frac{\partial ((1-t) \log(1-y))}{\partial y}$$

$$= -\frac{t}{y} - \frac{(1-t)}{1-y} (-1)$$

$$= -\frac{t}{y} + \frac{1-t}{1-y}$$

$$w_j \leftarrow w_j - \alpha \frac{\partial \mathcal{J}}{\partial w_j} \text{ 代入}$$

$$= w_j - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) x_j^{(i)}$$

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial w_j} = - \left( -\frac{t}{y} + \frac{1-t}{1-y} \right) \cdot y \cdot (1-y) \cdot x_j$$

$$= -t(1-y)x_j + (1-t)yx_j = -tx_j + tyx_j + yx_j - tyx_j = (y-t)x_j$$

$$\frac{\partial y}{\partial z} = \frac{\partial}{\partial z} \left( \frac{1}{1+e^{-z}} \right) = \frac{0 - 1 \cdot e^{-z} \cdot (-1)}{(1+e^{-z})^2}$$

$$= \frac{1 \times e^{-z}}{(1+e^{-z})(1+e^{-z})} = y(1-y)$$

$$\frac{\partial z}{\partial w_j} = \frac{\partial (w_j x_j)}{\partial w_j} = x_j$$

# Gradient Descent for Logistic Regression

---

## Comparison of gradient descent updates:

- Linear regression:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

- Logistic regression:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\alpha}{N} \sum_{i=1}^N (y^{(i)} - t^{(i)}) \mathbf{x}^{(i)}$$

- Not a coincidence! These are both examples of **generalized linear models**. But we won't go in further detail.
- Notice  $\frac{1}{N}$  in front of sums due to averaged losses. This is why you need smaller learning rate when cost is summed losses ( $\alpha' = \alpha/N$ ).

# Multiclass Classification

---

多类分类

Softmax 回归

Multiclass Classification and Softmax Regression

# Overview

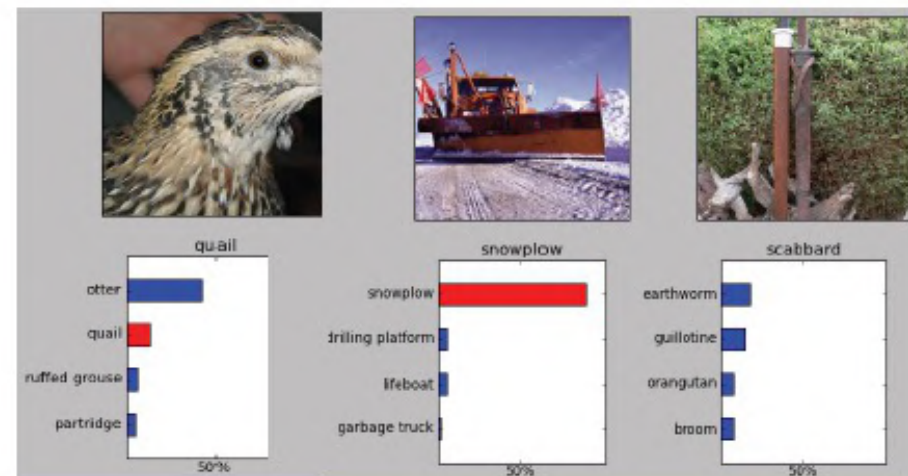
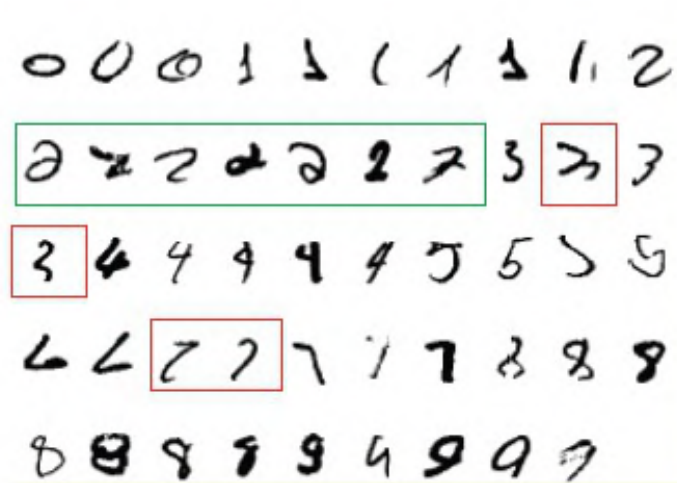
---

- **Classification**: predicting a discrete-valued target
  - **Binary classification**: predicting a binary-valued target
  - **Multiclass classification**: predicting a discrete-(>2)-valued target
- Examples of multi-class classification
  - Predict the value of a handwritten digit
  - Classify e-mails as spam, travel, work, personal

# Multiclass Classification

---

- Classification tasks with more than two categories:



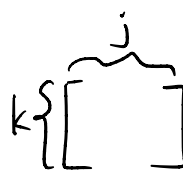
## Multiclass Classification

---

- Targets form a discrete set  $\{1, \dots, K\}$ . 目标集会包含两种以上的类别
- It's often more convenient to represent them as **one-hot vectors**, or a **one-of-K encoding**: 独热编码

$$\mathbf{t} = (\underbrace{0, \dots, 0, 1, 0, \dots, 0}_{\text{entry } k \text{ is } 1}) \in \mathbb{R}^K$$

创建一个长度为k的向量, 向量中只有一个元素是“热”的 (值为1), 其他所有元素都是“冷”的 (值为0). 1所处的位置就是当前类别.



$$W = \begin{matrix} \text{输出类别数量 (k)} \\ \text{(行)} \end{matrix} \times \begin{matrix} \text{输入特征数量 (D)} \\ \text{(列)} \end{matrix}$$

## Multiclass Linear Classification

- We can start with a linear function of the inputs. 对于多类别分类, 需要  $k$  个权重向量, 形成一个权重矩阵  $W$ .
- Now there are  $D$  input dimensions and  $K$  output dimensions, so we need  $K \times D$  weights, which we arrange as a **weight matrix**  $W$ .
- Also, we have a  $K$ -dimensional vector  $\mathbf{b}$  of biases. 我们也需要  $k$  个 bias  $b$ , 组成偏置向量  $\mathbf{b}$ .
- A linear function of the inputs:

$$z_k = \sum_{j=1}^D w_{kj} x_j + b_k \quad \text{for } k = 1, 2, \dots, K$$

- We can eliminate the bias  $\mathbf{b}$  by taking  $\mathbf{W} \in \mathbb{R}^{K \times (D+1)}$  and adding a dummy variable  $x_0 = 1$ . So, vectorized:

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b} \quad \text{or with dummy } x_0 = 1 \quad \mathbf{z} = \mathbf{W}\mathbf{x}$$

## Multiclass Linear Classification

---

- How can we turn this linear prediction into a **one-hot prediction**?
- We can interpret the magnitude of  $z_k$  as a measure of how much the model prefers  $k$  as its prediction.
- If we do this, we should set

$$y_i = \begin{cases} 1 & i = \arg \max_k z_k \\ 0 & \text{otherwise} \end{cases}$$

使用  $\arg \max$  : Argument of the maximum

$\arg \max(z)$ : 返回的是最大值所在的位置, 也就是最大数所在索引 (从1开始).

但是  $\arg \max$  属于硬预测

argmax 只能告诉你模型认为哪个类别的可能性最大,但无法告诉你它到底有多确定,也无法告诉你其他类别的可能性有多大

每类别的预测概率  $y_k$  都在 0 到 1 之间  
所有类别的预测概率加起来总和必须等于 1.

## Softmax Regression

- We need to soften our predictions for the sake of optimization.
- We want soft predictions that are like probabilities, i.e.,  $0 \leq y_k \leq 1$  and  $\sum_k y_k = 1$ .
- A natural activation function to use is the **softmax function**, a multivariable generalization of the logistic function:

$$y_k = \text{softmax}(z_1, \dots, z_K)_k = \frac{e^{z_k}}{\sum_{k'} e^{z_{k'}}}$$

分子  $e^{z_k}$ :  $\exp()$  函数有两个好处:

- ① 它可以把任何实数,无论正负变成正数
- ② 它可以拉大分数之间的差距,让大的更大.

- Outputs can be interpreted as probabilities (positive and sum to 1)
- If  $z_k$  is much larger than the others, then  $\text{softmax}(\mathbf{z})_k \approx 1$  and it behaves like argmax.

分母:  $\sum e^{z_k}$ : 把所有类别经过  $\exp()$  转换后的值加起来,作为一个总能量或归一化的常数.

## Softmax Regression

使用交叉熵损失来评价这个概率分布预测好不好

- If a model outputs a vector of class probabilities, we can use cross-entropy as the loss function:

$$\mathcal{L}_{\text{CE}}(\mathbf{y}, \mathbf{t}) = - \sum_{k=1}^K t_k \log y_k \\ = -\mathbf{t}^T (\log \mathbf{y}),$$

$t_k$  one-hot 向量, 只有1个  $t_k$  是1, 其他是0.

例如真实类别是第2个  $t_k = [0, 1, 0]$ .

那么损失值就直接是  $-\log y_2$ .

这样只关心模型为正确类别所预测的概率值

where the log is applied elementwise.

如果模型预测正确类别的概率  $y_2$  很高 损失  $-\log(0.9)$  很小

反之, 如果预测正确类别的概率  $y_2$  很小 损失  $-\log(0.1)$  很大

- Just like with logistic regression, we typically combine the softmax and cross-entropy into a **softmax-cross-entropy** function.

模型会给予很大的惩罚.

## Softmax Regression

---

- Softmax regression (with dummy  $x_0 = 1$ ):

$$\begin{aligned} \text{线性分数 } \mathbf{z} &= \mathbf{W}\mathbf{x} \\ \text{概率输出 } \mathbf{y} &= \text{softmax}(\mathbf{z}) \\ \text{损失函数 } \mathcal{L}_{\text{CE}} &= -\mathbf{t}^\top (\log \mathbf{y}) \end{aligned}$$

- Gradient descent updates can be derived for each row of  $\mathbf{W}$ :

$$\frac{\partial \mathcal{L}_{\text{CE}}}{\partial \mathbf{w}_k} = \frac{\partial \mathcal{L}_{\text{CE}}}{\partial z_k} \cdot \frac{\partial z_k}{\partial \mathbf{w}_k} = (y_k - t_k) \cdot \mathbf{x}$$

$$\mathbf{w}_k \leftarrow \mathbf{w}_k - \alpha \frac{1}{N} \sum_{i=1}^N (y_k^{(i)} - t_k^{(i)}) \mathbf{x}^{(i)}$$

(预测值 - 真实值) · 输入值

- Similar to linear/logistic reg (no coincidence) (verify the update)

$$\left(\frac{f(x)}{g(x)}\right)' = \frac{f'g - fg'}{g^2}$$

Prove the gradient ?

$$L_{CE} = -\sum_{k=1}^K t_k \log y_k, \quad y_k = \frac{e^{z_k}}{\sum_{k'} e^{z_{k'}}}$$

$$\text{when } T \neq k, \quad \frac{\partial y_T}{\partial z_k} = \frac{\partial}{\partial z_k} \left( \frac{e^{z_T}}{\sum_{k'} e^{z_{k'}}} \right)$$

$$\frac{\partial L_{CE}}{\partial w_k} = \underbrace{\frac{\partial L_{CE}}{\partial y_T}}_{(1)} \cdot \underbrace{\frac{\partial y_T}{\partial z_k}}_{(2)} \cdot \underbrace{\frac{\partial z_k}{\partial w_k}}_{(3)}$$

$$(1) \quad \frac{\partial L_{CE}}{\partial y_T} = \frac{\partial}{\partial y_T} \left( -\sum_{k=1}^K t_k \log y_k \right) = -\frac{t_T}{y_T} = -\frac{1}{y_T} \quad (\text{当 } T=k \text{ 时}).$$

$$(1) \times (2) \quad \begin{cases} T=k & -\frac{1}{y_T} \cdot y_k (1-y_k) = y_k - 1 \\ & = y_k - t_k \end{cases}$$

$$(2) \quad \text{when } T=k : \quad \frac{\partial y_T}{\partial z_k} = \frac{\partial y_k}{\partial z_k} = \frac{\partial}{\partial z_k} \left( \frac{e^{z_k}}{\sum_{k'} e^{z_{k'}}} \right) \quad (\text{when } k'=z_k)$$

$$T \neq k \quad -\frac{1}{y_T} \cdot (1-y_T) y_k = y_k - 0 \\ = y_k - t_k$$

$$\begin{aligned} &= \frac{e^{z_k} \cdot \sum_{k'} e^{z_{k'}} - e^{z_k} \cdot e^{z_k}}{(\sum_{k'} e^{z_{k'}})^2} \\ &= \frac{e^{z_k} \cdot (\sum_{k'} e^{z_{k'}} - e^{z_k})}{(\sum_{k'} e^{z_{k'}}) \cdot (\sum_{k'} e^{z_{k'}})} \end{aligned}$$

$y_k$ 
 $(1-y_k)$

$$(3) \quad \frac{\partial z_k}{\partial w_k} = \frac{\partial (w_k \cdot x)}{\partial w_k} = x$$

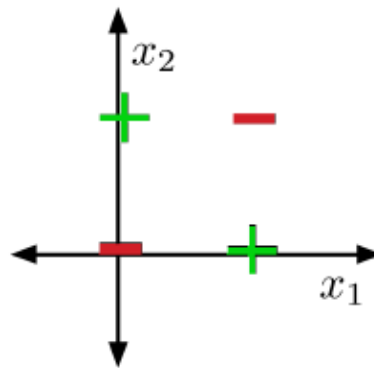
$$\therefore \frac{\partial L_{CE}}{\partial w_k} = (y_k - t_k) \cdot x$$

$$= y_k (1 - y_k)$$

## Limits of Linear Classification

---

Some datasets are not linearly separable, e.g. **XOR**

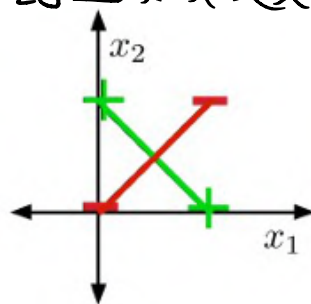


Visually obvious, but how to show this?

## Limits of Linear Classification

Showing that XOR is not linearly separable (proof by contradiction) 反证法,

- If two points lie in a half-space, line segment connecting them also lie in the same half-space. 线性分类器的直线会把空间分成两个半空间。如果两个点在一个半空间，那么连接它们的线段也一定完全在这个半空间。
- Suppose there were some feasible weights (hypothesis). If the positive examples are in the positive half-space, then the green line segment must be as well. 先假设存在这样一条完美的分割线。
- Similarly, the red line segment must lie within the negative half-space. 分割的线，如果存在分割线，那么绿色的点在线段一侧，红色的点在线段另一侧。但是绿色点的连接线和红色点的连接线交叉了，这个交点不可能只位于分割线的一侧。这是一个无法解决的矛盾。



- But the intersection can't lie in both half-spaces. Contradiction!

## Limits of Linear Classification

当一个问题在低维空间不可分时, 可以尝试映射到一个高维

- Sometimes we can overcome this limitation using **feature maps**, just like for linear regression. E.g., for **XOR**:

$$\psi(\mathbf{x}) = \begin{pmatrix} x_1 \\ x_2 \\ x_1 x_2 \end{pmatrix}$$

| $x_1$ | $x_2$ | $\psi_1(\mathbf{x})$ | $\psi_2(\mathbf{x})$ | $\psi_3(\mathbf{x})$ | $t$    |
|-------|-------|----------------------|----------------------|----------------------|--------|
| 0     | 0     | 0                    | 0                    | 0                    | 0 -0.5 |
| 0     | 1     | 0                    | 1                    | 0                    | 1 0.5  |
| 1     | 0     | 1                    | 0                    | 0                    | 1 0.5  |
| 1     | 1     | 1                    | 1                    | 1                    | 0 -0.5 |

$$\begin{aligned} w_1 &= 1 \\ w_2 &= 1 \\ w_3 &= -2 \\ d &= -0.5 \end{aligned}$$

- This is linearly separable. (Try it!)

$$z_1 + z_2 - 2z_3 - 0.5 = 0 \quad \text{可以将正类点和负类点分开}$$

## Next time...

---

Feature maps are hard to design well, so next time we'll see how to *learn* nonlinear feature maps directly using neural networks...

